ARTIFICIAL INTELLIGENCE LAB

11. Map Coloring to Implement a Constraint Satisfaction Problem (CSP)

Aim

To implement map coloring as a Constraint Satisfaction Problem using backtracking, ensuring adjacent regions have different colors.

Algorithm

1. Represent regions as variables and available colors as domains.
2. Represent neighboring regions as constraints.
3. Assign a color to an uncolored region.
4. Check that no neighboring region has the same color.
5. If a conflict occurs, backtrack and try another color.
6. Print the assignment if all regions are colored successfully.

Program (Python)

```
def is_safe(region, color, assignment, graph):
    for neighbor in graph[region]:
        if assignment.get(neighbor) == color:
            return False
    return True

def solve_map_coloring(graph, colors, assignment=None):
    if assignment is None:
        assignment = {}

    if len(assignment) == len(graph):
        return assignment

    # Choose the next unassigned region
    region = next(r for r in graph if r not in assignment)

    for color in colors:
        if is_safe(region, color, assignment, graph):
            assignment[region] = color
            result = solve_map_coloring(graph, colors, assignment)
            if result:
                return result
            del assignment[region]
    return None

if __name__ == "__main__":
    graph = {
        "A": ["B", "C"],
        "B": ["A", "C", "D"],
        "C": ["A", "B", "D"],
        "D": ["B", "C"]
    }
    colors = ["Red", "Green", "Blue"]
    solution = solve_map_coloring(graph, colors)
```

```
    if solution:
        print("Map coloring solution:")
        for region, color in solution.items():
            print(f"{region}: {color}")
    else:
        print("No solution exists.")
```

Sample Output

```
= RESTART: C:/Users/admin/Docu
Map coloring solution:
A: Red
B: Green
C: Blue
D: Red
```

Result

The map-coloring CSP is solved using backtracking, assigning different colors to adjacent regions.

12. Tic-Tac-Toe Game

Aim

To implement a simple two-player Tic-Tac-Toe game in Python.

Algorithm

7. Initialize a 3 × 3 board.
8. Display the board and ask the current player for a position from 1 to 9.
9. Validate the position and mark the cell.
10. Check for a winning row, column, or diagonal, or a draw.
11. Alternate players until the game ends.

Program (Python)

```
def display(board):
    print()
    for i in range(0, 9, 3):
        print(" | ".join(board[i:i+3]))
        if i < 6:
            print("--+---+--")
    print()

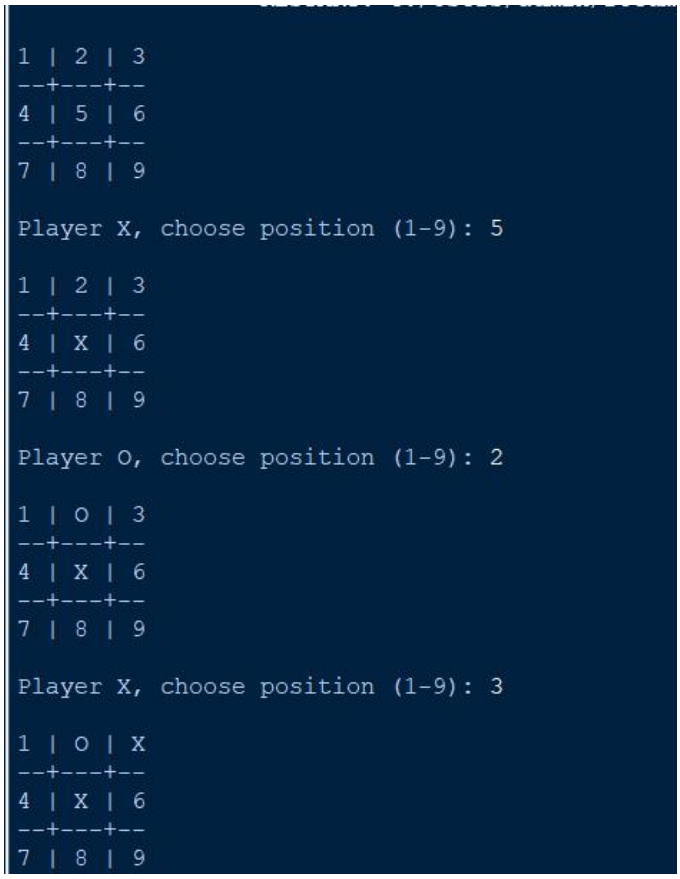
def winner(board, mark):
    lines = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]
    return any(all(board[i] == mark for i in line) for line in lines)

def main():
    board = [str(i) for i in range(1, 10)]
    player = "X"
    for _ in range(9):
        display(board)
        try:
```

```
            pos = int(input(f"Player {player}, choose position (1-9): ")) - 1
        except ValueError:
            print("Enter a number.")
            continue
        if not 0 <= pos < 9 or board[pos] in ("X", "O"):
            print("Invalid or occupied position.")
            continue
        board[pos] = player
        if winner(board, player):
            display(board)
            print(f"Player {player} wins!")
            return
        player = "O" if player == "X" else "X"
    display(board)
    print("It's a draw!")

if __name__ == "__main__":
    main()
```

Sample Output




```
... (continues until a win or draw)
```

Result

The program allows two players to play Tic-Tac-Toe and identifies a win or draw.

13. Minimax Algorithm for Gaming

Aim

To implement the Minimax algorithm for choosing the best move in a small game tree.

Algorithm

12. If a node is terminal, return its utility value.
13. At a maximizing node, choose the maximum child value.
14. At a minimizing node, choose the minimum child value.
15. Return the best move and its score.

Program (Python)

```
def minimax(tree, maximizing):
    if isinstance(tree, int):
        return tree
    values = [minimax(child, not maximizing) for child in tree]
    return max(values) if maximizing else min(values)

if __name__ == "__main__":
    # Leaves are utility values; levels alternate MAX and MIN.
    game_tree = [
        [3, 5, 2],
        [9, 1, 4],
        [6, 7, 8]
    ]
    scores = [minimax(branch, False) for branch in game_tree]
    best_move = max(range(len(scores)), key=lambda i: scores[i])
    print("Branch scores:", scores)
    print("Best move: Branch", best_move + 1)
    print("Best score:", scores[best_move])
```

Sample Output

```
========== RESTART: C:/
Branch scores: [2, 1, 6]
Best move: Branch 3
Best score: 6
```

Result

The Minimax algorithm evaluates possible game outcomes and selects the move with the best guaranteed score.

14. Alpha-Beta Pruning Algorithm for Gaming

Aim

To implement Alpha-Beta pruning to reduce the number of game-tree nodes evaluated by Minimax.

Algorithm

16. Initialize alpha to negative infinity and beta to positive infinity.
17. At MAX nodes, update alpha with the best value found.
18. At MIN nodes, update beta with the best value found.
19. Stop exploring a branch when alpha is greater than or equal to beta.
20. Return the best value.

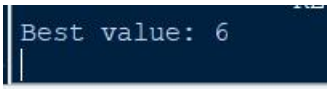
Program (Python)

```
def alphabeta(node, depth, alpha, beta, maximizing):
    if depth == 0 or isinstance(node, int):
        return node

    if maximizing:
        value = float("-inf")
        for child in node:
            value = max(value, alphabeta(child, depth - 1,
                                         alpha, beta, False))
            alpha = max(alpha, value)
            if alpha >= beta:
                break
        return value
    else:
        value = float("inf")
        for child in node:
            value = min(value, alphabeta(child, depth - 1,
                                         alpha, beta, True))
            beta = min(beta, value)
            if beta <= alpha:
                break
        return value

if __name__ == "__main__":
    tree = [[3, 5, 2], [9, 1, 4], [6, 7, 8]]
    best = alphabeta(tree, 2, float("-inf"), float("inf"), True)
    print("Best value:", best)
```

Sample Output



Result

Alpha-Beta pruning returns the same optimal value as Minimax while pruning branches that cannot affect the decision.

15. Decision Tree

Aim

To train and test a Decision Tree classifier using a sample dataset with scikit-learn.

Algorithm

21. Import the DecisionTreeClassifier and a sample dataset.
22. Split the dataset into training and testing sets.
23. Train the decision tree on the training data.
24. Predict labels for the test data.
25. Print predictions and classification accuracy.

Program (Python)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import accuracy_score

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2,
    random_state=42, stratify=data.target
)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("Predictions:", predictions)
print("Accuracy:", round(accuracy_score(y_test, predictions) * 100, 2), "%")
```

Sample Output

```
Predictions: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 1 2 2 1 0 2 0]
Accuracy: 93.33 %
```

Result

The Decision Tree model is trained on the Iris dataset and evaluated using test-set accuracy.

16. Feedforward Neural Network

Aim

To implement a basic feedforward neural network for classifying the Iris dataset using scikit-learn.

Algorithm

26. Load and split the Iris dataset.
27. Standardize input features.
28. Create a multilayer perceptron with one or more hidden layers.
29. Train the network using the training data.
30. Predict test labels and calculate accuracy.

Program (Python)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2,
    random_state=42, stratify=data.target
)

model = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(10,), max_iter=2000,
                  random_state=42)
```

```
)
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("Predictions:", predictions)
print("Accuracy:", round(accuracy_score(y_test, predictions) * 100, 2), "%")
```

Sample Output

```
Predictions: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 2 1 0 2 0]
Accuracy: 96.67 %
```

Result

The feedforward neural network is trained and evaluated on the Iris dataset.